



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.7: Cryptocurrency Cryptography

Nadim Kobeissi

<https://appliedcryptography.page>

Section 1

Bitcoin

The problem: trust in digital commerce

- Online commerce relies entirely on financial institutions as trusted third parties.
- Banks, payment processors, credit card companies mediate every transaction.
- This trust-based model has fundamental limitations:
 - Intrinsic dependence on centralized parties.
 - Financial institutions must mediate disputes.
 - Increases transaction costs significantly.

Weaknesses of the trust-based model

- **High transaction costs:** Mediation costs make micropayments impractical.
- **Reversibility spreads distrust:** Merchants need extensive customer information.
- **Fraud is inevitable:** Some percentage of fraud accepted as cost of business.
- **No digital cash equivalent:** Physical cash works person-to-person, but no digital analog exists.

Peer-to-peer electronic cash

- Replace trust with cryptographic proof.
- Enable direct transactions between parties without intermediaries.
- Key properties needed:
 - Computationally impractical to reverse (protects sellers).
 - Optional escrow mechanisms (protects buyers).
 - No reliance on third parties.

Core challenge: double-spending

- Digital information can be copied perfectly.
- Without a central authority, how do we prevent spending the same coin twice?
- Traditional solution: Central mint/bank tracks all transactions.
- Problem: This reintroduces the trusted third party.

Bitcoin's solution

- Peer-to-peer distributed timestamp server.
- Generates computational proof of transaction order.
- Security assumption: Honest nodes control majority of CPU power.
- Key innovations:
 - Proof-of-work consensus mechanism.
 - Blockchain as public ledger.
 - Economic incentives for honest behavior.

Historical context

- Published October 31, 2008 by Satoshi Nakamoto (pseudonym).
- Built on decades of cryptographic research:
 - Digital signatures (ECDSA)
 - Hashcash proof-of-work (Adam Back, 1997)
 - Cryptographic timestamps (Haber & Stornetta, 1991)
 - B-money and bit gold proposals (Wei Dai, Nick Szabo)
- First practical solution to Byzantine Generals Problem for money.

The Byzantine Generals Problem

- Classical problem in distributed computing (Lamport, Shostak, Pease, 1982).
- Scenario: Byzantine generals must coordinate attack on a city.
 - Generals can only communicate via messengers.
 - Some generals may be traitors sending false messages.
 - Need consensus: all loyal generals execute same plan.
- Core challenge: How to achieve reliable consensus when:
 - Communication channels are unreliable.
 - Some participants may be malicious.
 - No central authority to trust.
- **Bitcoin's breakthrough:** First practical solution for digital money using proof-of-work.

Section 1

Bitcoin

Subsection 1.1

Blockchains

Let's start with the basics: What's a blockchain?

Think of it like a notebook:

- Pages = blocks
- Each page lists transactions
- Pages linked together in order
- Can't tear out or reorder pages

But it's distributed:

- Everyone has a copy
- No single owner
- Need agreement to add pages

The cryptographic glue:

- Each block contains:
 - List of transactions
 - Hash of previous block
 - Timestamp
 - Special "proof" (we'll see why!)

Key insight

The hash links make history tamper-proof
- change one block and all subsequent hashes break!

The challenge: Who gets to add the next block?

In a decentralized system with no leader, how do we decide?

Can't use traditional voting:

- One-person-one-vote?
 - How to verify identities?
 - Attacker creates 1000 fake identities
 - This is a "Sybil attack"
- Central authority decides?
 - Defeats the whole point!
 - Back to trusting intermediaries

Bitcoin's brilliant solution:

- Make it expensive to add blocks
- But anyone can try!
- Winners prove they did work
- Can't fake the work

This is Proof-of-Work

Replace "one identity = one vote" with
"one CPU cycle = one vote"

Section 1

Bitcoin

Subsection 1.2

Proof-of-Work

Proof-of-Work: A puzzle anyone can solve (but slowly)

The basic idea:

- To add a block, solve a puzzle
- Puzzle is hard to solve
- But easy to verify solution
- Like Sudoku:
 - Hard to find solution
 - Easy to check if correct

Why this works:

- Can't create fake identities to cheat
- Each attempt requires real computation
- Computational power = physical resources
- Can't pretend to have more than you do

Key property

You prove you spent resources without revealing anything sensitive

The hash lottery: Bitcoin's specific puzzle

The puzzle:

- Take your block data
- Add a random number (nonce)
- Hash everything: $H = \text{SHA256}(\text{SHA256}(\text{block} + \text{nonce}))$
- Check if H starts with many zeros
- If not, try different nonce
- Repeat until you find one!

Example hashes:

- 0000000000000000000000007b ...
 - Winner! (19 leading zeros)
- 5fec4ba9f3c2d8a1b9e7 ...
 - No good, try again

Why hashing?

Each attempt is independent - can't get "closer" to solution, only keep trying

How hard is this puzzle really?

Let's look at real Bitcoin numbers

Current difficulty (2024):

- Need ≈ 19 -20 leading hex zeros
- That's ≈ 76 -80 leading zero bits
- Finding a hash smaller than $\approx 2^{180}$ out of 2^{256} possible
- Probability per attempt: $\approx \frac{1}{2^{76}}$, astronomically small!

In practice:

- Network tries $\approx 600 \times 10^{18}$ hashes per second
- That's 600 exahashes/second!
- All to find one valid block
- Every 10 minutes on average

Mind-boggling scale

More computations than there are grains of sand on Earth, every 10 minutes!

Difficulty adjustment: The technical details

What Bitcoin measures:

- Every 2016 blocks
- Calculate actual time taken
- Target: 20,160 minutes (2 weeks)
- Compare actual vs. target

The adjustment formula:^a

- $$\text{New difficulty} = \text{Old difficulty} \times \frac{\text{Target time}}{\text{Actual time}}$$
- Blocks too fast? Difficulty $\uparrow$
- Blocks too slow? Difficulty $\downarrow$

Example calculation:

- 2016 blocks in 10 days
 - Too fast! (should be 14 days)
 - Ratio = $14/10 = 1.4$
 - Increase difficulty by 40%
- 2016 blocks in 20 days
 - Too slow!
 - Ratio = $14/20 = 0.7$
 - Decrease difficulty by 30%

Safety limit

Maximum adjustment: 4x up or down per period (prevents manipulation)

^a<https://learnmeabitcoin.com/beginners/guide/difficulty/>

The mining process: Finding the golden ticket

What miners actually do:

1. Collect pending transactions
2. Create block with:
 - Transactions
 - Hash of previous block
 - Timestamp
 - Nonce = 0
3. Compute hash
4. Check: enough zeros?
5. If no: nonce++, try again
6. If yes: Broadcast to network!

Why call it “mining”?

- Like prospecting for gold
- Search through possibilities
- Most attempts fail
- Finding one is valuable
- Rewards early participants

The reward

Winner gets to create new bitcoins (currently 3.125 BTC $\approx$ \$200K) plus transaction fees

Why proof-of-work solves our problems

Remember the challenges? PoW solves them all!

Problem 1: Sybil attacks

- Can't fake computational work
- 1000 fake identities with 1 hash/sec each = same as 1 identity with 1000 hash/sec
- No advantage to splitting!

Problem 2: Who decides ordering?

- Whoever solves puzzle first
- It's a random lottery
- But weighted by hashpower

Problem 3: Double-spending

- To rewrite history:
 - Must redo all the work
 - For that block + all after
 - Faster than honest miners
- Cost grows exponentially!

Security guarantee

As long as honest miners have >50% of computing power, the system is secure

The elegant lottery mechanism

Each hash is a lottery ticket:

- Every attempt has same probability
- Can't get "closer" to winning
- No way to predict winner
- Can't save progress

This creates fairness:

- 10% of hashpower = 10% chance to win
- Proportional to investment
- No special advantages

Why randomness matters:

- Unpredictable who wins next
- Can't collude in advance
- Everyone racing simultaneously
- First to find solution wins

Beautiful property

Pure democracy of computation - the most physically real form of voting!

The difficulty adjustment: Bitcoin's thermostat

The problem:

- More miners join → blocks too fast
- Miners leave → blocks too slow
- Want consistent 10-minute blocks

The solution:

- Every 2016 blocks (≈ 2 weeks)
- Measure actual time taken
- Adjust difficulty up or down
- Like a thermostat!

How adjustment works:

- Blocks coming too fast?
 - Require more leading zeros
 - Makes puzzle harder
- Blocks coming too slow?
 - Require fewer leading zeros
 - Makes puzzle easier

Result

Stable 10-minute block time for 15+ years, despite massive hashpower growth!

The economics: Why miners keep mining

Mining costs:

- Specialized hardware (ASICs)
 - \$2,000-10,000 per machine
- Electricity (biggest cost!)
 - \$30,000-100,000 per machine/year
- Cooling, facilities, maintenance

Mining rewards:

- Block subsidy: 3.125 BTC
 - Halves every 4 years
 - Started at 50 BTC in 2009
- Transaction fees
 - Vary by network congestion

Equilibrium

Total mining cost $\approx$ Total rewards

This ensures appropriate security!

Why miners keep mining despite halvings

The concern:

- Block reward keeps halving
- Started at 50 BTC (2009)
- Now 3.125 BTC^a
- Eventually approaches zero
- Won't miners quit?

^a<https://learnmeabitcoin.com/technical/mining/block-reward/>

Why it works:

- Bitcoin price tends to rise
 - 50 BTC in 2009: \$0
 - 3.125 BTC in 2024: \$200K
- Transaction fees increase
 - More users = more fees
 - Will eventually dominate
- Difficulty adjusts down
 - If miners leave, easier to mine
 - Self-regulating system

Long-term transition

Gradually shifting from subsidy-funded to fee-funded security model

The environmental elephant in the room

All this computation uses massive energy

The scale:

- Peak (2021): ≈ 180 TWh/year
- More than Argentina's total consumption
- Individual facilities use 700 MW
 - Powers 500,000 homes!
- Kazakhstan: 18% of global mining
 - Caused power shortages
 - Government restrictions

The debate:

- **Critics say:**
 - Wasteful computation
 - Environmental disaster
 - Unsustainable
- **Supporters say:**
 - Secures \$1T+ of value
 - Uses stranded/renewable energy
 - Alternative to gold mining

Alternative PoW designs: Can we do better?

The ASIC problem:

- Specialized chips dominate Bitcoin
- Regular people can't compete
- Centralization in few hands
- Is this the vision?

Attempted solutions:

- Memory-hard algorithms
- Random VM execution
- Frequently changing puzzles

Example: Ethereum's old PoW

- Required 4 GB dataset in RAM
- Random memory access
- ASICs less advantageous
- GPUs competitive

The trade-off

More democratic access vs. more complex verification and potential security issues

Section 1

Bitcoin

Subsection 1.3

From Blocks to Blockchain

How blocks form an unbreakable chain

Each block contains:

- List of transactions
- Timestamp
- Hash of previous block
- Nonce (the solution!)

This creates a chain:

- Block N points to Block N-1
- Which points to Block N-2
- All the way to Block 0 (genesis)

Why it's tamper-proof:

- Change any transaction
- Block hash changes
- Next block's "previous hash" invalid
- Must redo all subsequent work
- Must outpace honest miners

Security grows over time

Each new block makes all previous blocks more secure!

Temporary forks: When two miners win simultaneously

What happens:

- Miner A finds block at height 100
- Miner B finds different block at height 100
- Both valid!
- Half the network sees A first
- Half sees B first
- Temporary fork

Resolution:

- Both branches grow
- Next block extends one branch
- That branch becomes longer
- Everyone switches to longer chain
- Losing block becomes “orphaned”

Self-healing

The system automatically converges without any coordination!

The longest chain rule: Democracy of hashpower

Simple rule: Always follow the longest valid chain

Why “longest”?

- Longest = most cumulative work
- Most work = most hashpower
- Most hashpower = majority
- Majority = honest (we hope!)

What this achieves:

- Automatic consensus
- No voting needed
- No coordination required
- Objective measurement (work)
- Economic incentives align

This is Nakamoto consensus - the heart of Bitcoin!

The 51% attack: The one big vulnerability

What if attacker gets >50% hashpower?

- Can build longer chain privately
- Eventually overtakes honest chain
- Can double-spend their own transactions
- Can censor other transactions

But CANNOT:

- Steal others' coins (need private keys)
- Create coins from nothing (invalid blocks rejected)

Why this is unlikely:

- Requires enormous investment
- Would crash Bitcoin price
- Attack destroys attacker's investment
- More profitable to mine honestly!

Economic security

The cost to attack exceeds any possible gain - this is game-theoretic security!

Section 1

Bitcoin

Subsection 1.4

Bitcoin Transactions and Ownership

How Bitcoin defines ownership

No accounts, only keys:

- Private key: Random 256-bit number
 - Your secret
 - Proves ownership
 - Never share!
- Public key: Derived from private
 - Cryptographic magic (ECDSA)
 - Can verify signatures
- Address: Hash of public key
 - What you share
 - Like a bank account number

The mailbox analogy:

- Address = mailbox number
 - Everyone can see it
 - Anyone can send to it
- Private key = mailbox key
 - Only you have it
 - Only you can retrieve mail

Critical rule

Not your keys, not your coins!

The UTXO model: Bitcoin's unique approach

Not account balances!

- Bitcoin tracks **Unspent Transaction Outputs**
- Have a \$20 and a \$5 bill
- That's 2 UTXOs totaling \$25

Example transaction:

- Want to send \$15
- Must spend entire \$20 bill
- Recipient gets \$15
- You get \$5 change
- New UTXOs created!

Bitcoin example:

- Have 5 BTC UTXO
- Want to send 2 BTC
- Transaction:
 - Input: 5 BTC (consumed)
 - Output 1: 2 BTC to recipient
 - Output 2: 2.9 BTC change to you
 - Implicit fee: 0.1 BTC (input — outputs)

Why this model?

Prevents double-spending more naturally than account balances!

Electronic coins as chains of signatures

A coin is just a chain of digital signatures

To transfer a coin:

1. Take previous transaction
2. Add recipient's public key
3. Sign with your private key
4. This becomes new transaction

Anyone can verify:

- Check all signatures in chain
- Each signature proves ownership at that time

The signature proves

- You owned the coin
- You authorized the transfer
- Transaction not tampered with

But one problem...

- What prevents signing twice?
- Could send same coin to two people!
- This is the double-spend problem

Creating new coins: The coinbase transaction

Where do bitcoins come from?

- Every block has first transaction
- Called “coinbase transaction”
 - Not the exchange company!
 - Bitcoin protocol term
- Has no inputs
- Creates coins from nothing!
- Goes to the miner

Bitcoin's monetary policy:

- Started at 50 BTC per block (2009)
- Halves every 210,000 blocks
 - Roughly every 4 years
- Currently: 3.125 BTC
- Next halving: 1.5625 BTC (2028)
- Eventually: 0 (around year 2140)
- Maximum supply: 21 million BTC

Section 1

Bitcoin

Subsection 1.5

Solving the Double-Spend Problem

The core problem: Order without clocks

How do we agree which transaction came first?

Why this is hard:

- No central timestamp server
- Network delays vary
- No global clock
- Attacker can lie about timing
- Can't trust any single party

Traditional solution:

- Bank keeps the ledger
- Bank decides order
- Everyone trusts the bank
- But we want to avoid that!

Need

Cryptographic proof of order that everyone agrees on

Bitcoin's solution: A chain of timestamps

The innovation:

- Bundle transactions into blocks
- Each block has timestamp
- Each block hashes previous block
- Chain of timestamps!

Why this works:

- Can't backdate a block
 - Would need to redo all work after it
- Each block reinforces previous ones
- History becomes progressively harder to change

But who creates timestamps?

- This is where PoW comes in!
- Mining creates the timestamp
- Difficulty prevents spam
- Proof-of-work proves timestamp

Beautiful circularity

PoW enables timestamping
Timestamping prevents double-spend
PoW rewards pay for security

Preventing double-spend with proof-of-work

Attacker tries to double-spend:

- Sends transaction to merchant
- Merchant sees transaction in block
- Attacker secretly mines alternative chain
 - Different transaction spending same coins
 - Tries to overtake honest chain

Why it fails:

- Honest miners keep extending main chain
- Attacker must mine faster than all honest miners combined
- Probability decreases exponentially
- After 6 blocks: effectively impossible

Wait for confirmations

More blocks = more security. 6 confirmations = standard for high-value transactions

Transaction confirmation in practice

How merchants handle this:

- 0 confirmations: High risk
 - Coffee shop: OK
 - Car dealership: No way
- 1 confirmation: Some risk
 - ≈ 10 minutes wait
 - Acceptable for small amounts
- 6 confirmations: Standard
 - ≈ 1 hour wait
 - Used by exchanges

The math behind it:

- Attacker with 10% hashpower:
 - 1 block: 10% success
 - 2 blocks: 1% success
 - 6 blocks: 0.0001% success
- With 30% hashpower:
 - 6 blocks: $\approx 0.5\%$ success

Security grows

Each confirmation exponentially decreases attack probability!

Putting it all together: The full picture

How all the pieces work together

Transactions:

- Digital signatures prove ownership
- Spent outputs consumed
- New outputs created

Blocks:

- Bundle transactions together
- Include timestamp
- Link to previous block

Mining:

- Proof-of-work creates blocks
- Economic incentives align
- Difficulty adjusts automatically

Consensus:

- Longest chain wins
- Double-spends prevented
- Network converges

Result: Trustless digital cash!

Section 1

Bitcoin

Subsection 1.6

Merkle Trees

The blockchain scaling problem

Problem: Bitcoin blocks are huge!

The challenge:

- Block can have 2000+ transactions
- Full block: $\approx 1\text{-}2$ MB
- New block every 10 minutes
- 850,000+ blocks so far
- Total: ≈ 500 GB!

What if you want to:

- Run Bitcoin on your phone?
- Verify a payment quickly?
- Not download 500 GB?

The question

Can you verify a transaction is in a block without downloading the entire block?

Merkle trees: Bitcoin's elegant solution

The insight:

- Don't need all transactions
- Just need to prove one transaction is included
- Use cryptographic hashing!

What Bitcoin does:

- Build **tree of hashes**
- Put single root in block header
- Can prove any transaction with small proof

The magic numbers:

- Block with 2000 txs: ≈ 1 MB
- Block header: 80 bytes
- Merkle proof: ≈ 400 bytes
- Reduction: $2500\times$ smaller!

Named after

Ralph Merkle (1979) - pioneer of public-key cryptography (worked closely with Diffie and Hellman)

Inside a Bitcoin block header

The 80-byte header:

- Version: 4 bytes
- Previous block hash: 32 bytes
- **Merkle root: 32 bytes**
- Timestamp: 4 bytes
- Difficulty: 4 bytes
- Nonce: 4 bytes

The Merkle root:

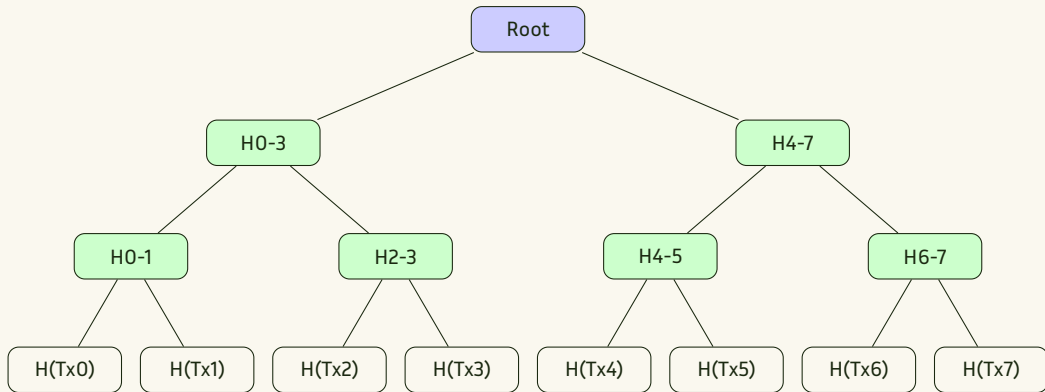
- Single hash
- Represents ALL transactions
- Change any transaction...
- Root completely changes!
- This is the commitment

Key property

The root cryptographically commits to every transaction in the block

Visual: Building the tree

Merkle tree with 8 transactions



Blue = What goes in block header | Green = Intermediate nodes | White = Leaves

How to build a Merkle tree: Step by step

Start with transactions:

- Hash each transaction
- These are the leaves
- Think: bottom of the tree

Build upward:

1. Pair adjacent hashes
2. Concatenate them
3. Hash the result
4. This is the parent node
5. Repeat until one hash left!

Example with 4 transactions:

- Level 0 (leaves):
 - $H(Tx0), H(Tx1)$
 - $H(Tx2), H(Tx3)$
- Level 1:
 - $H01 = \text{hash}(H(Tx0) \parallel H(Tx1))$
 - $H23 = \text{hash}(H(Tx2) \parallel H(Tx3))$
- Level 2 (root):
 - $\text{Root} = \text{hash}(H01 \parallel H23)$

The power: Logarithmic proofs

This is where Merkle trees shine!

To prove Tx2 is in the block:

- Don't need all 8 transactions
- Only need the "sibling path"
- Just 3 hashes:
 1. H(Tx3) - sibling
 2. H0-1 - uncle
 3. H4-7 - aunt
- Plus Tx2 itself

The math:

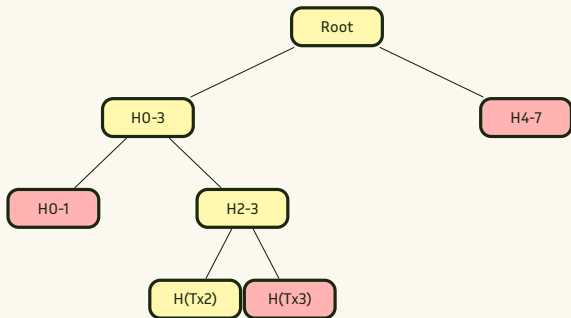
- 8 txs: Need 3 hashes
- 1,024 txs: Need 10 hashes
- 1 million txs: Need 20 hashes
- Path length = $\log_2(n)$

Incredible efficiency

Proof stays tiny even as block grows huge!

Visualizing the proof path

Proving Tx2 is in the block



- **Yellow = Compute these:** Start with Tx2, hash upward
- **Red = Proof provides these:** Just 3 hashes needed!

Step-by-step verification

What verifier has:

- Transaction Tx2
- Merkle proof: [H(Tx3), H0-1, H4-7]
- Block header with root

What verifier does:

1. Hash Tx2 $\rightarrow$ H(Tx2)
2. Combine with H(Tx3)
3. Hash $\rightarrow$ H2-3
4. Combine with H0-1
5. Hash $\rightarrow$ H0-3

6. Combine with H4-7
7. Hash $\rightarrow$ Computed Root
8. Compare with header

If they match

Tx2 is definitely in the block! Can't fake this without breaking SHA-256.

Data transferred:

Tx2 + 3 hashes = $\approx$ 400 bytes
Instead of 1 MB!

Where do Merkle proofs come from?

The question:

- You have the Merkle root
- You want to verify a transaction
- But you need sibling hashes
- Where do they come from?

The answer:

- **Full nodes have everything^a**
- **They store the entire block**
- **Including all transactions**
- Can compute any proof

How full nodes generate proofs:

1. Store all transactions in block
2. Rebuild the Merkle tree
3. Find transaction's position
4. Walk up tree, collect siblings
5. Return proof path

Key point

Full nodes can generate proofs for any transaction because they have the complete block data!

^a<https://learnmeabitcoin.com/technical/block/merkle-root/>

Full node verification workflow

When full node receives block:

1. Download all transactions
2. Build Merkle tree locally
3. Compute root
4. Compare with block header
5. If match: block valid!

To verify specific transaction:

- Already has full tree
- Extract sibling path
- Verify against root

Example scenario:

- Your wallet queries your full node
- “Is transaction X in block 850000?”
- Full node:
 1. Has block 850000 stored
 2. Locates transaction X
 3. Generates proof path
 4. Returns proof to wallet

Trust model

Full nodes trust no one - they verify everything themselves!

SPV: Simplified Payment Verification

Full node:

- Downloads everything
- Validates all transactions
- Stores full blockchain
- Requirement: ≈ 500 GB
- Time to sync: Days

Not practical for most users!

SPV client:

- Downloads only headers
- 80 bytes $\times$ 850K blocks
- Requirement: ≈ 60 MB
- Syncs in minutes
- Requests Merkle proofs as needed

8000 $\times$ smaller!

This is why Bitcoin works on phones!

How SPV wallets work

Setup:

1. Download all block headers
2. Verify PoW chain
3. Store locally (tiny!)
4. Now ready

When payment arrives:

1. Peer says: "You got paid!"
2. SPV wallet: "Prove it"
3. Peer sends Merkle proof
4. Wallet verifies against header

Security:

- Can verify inclusion
- Can't verify validity
 - Trust PoW majority
 - Full nodes reject invalid
- Wait for confirmations
- Connect to multiple peers

Trade-off

Less security than full node, but good enough for most use cases

Handling odd transaction counts

The problem:

- Binary trees need pairs
- What if 5 transactions?
- Can't pair the last one

Bitcoin's solution:

- Duplicate the last hash
- Tx4's hash appears twice
- Now we have 6 leaves
- Tree is complete!

Important distinction:

- Duplicate the *hash*
- NOT the transaction itself
- Transaction only in block once
- Prevents weird edge cases

Visual

Leaves: $H(Tx0)$, $H(Tx1)$, $H(Tx2)$, $H(Tx3)$,
 $H(Tx4)$, $H(Tx4)$

Why Merkle trees are brilliant

Efficiency:

- Logarithmic proof size
- Logarithmic verification time
- Enables light clients
- Mobile-friendly

Security:

- Tamper-evident
- Change any tx → root changes
- Based on collision resistance
- Strong as SHA-256

Privacy:

- Prove one transaction
- Don't reveal others
- Selective disclosure

Storage:

- Old nodes can prune
- Keep headers only
- Serve proofs on demand

Fundamental primitive

Merkle trees make Bitcoin scalable!

Security properties in detail

Tamper evidence:

- Modify any transaction
- Its hash changes
- Parent hash changes
- Propagates to root
- Root in header changes
- Breaks PoW!

Collision resistance:

- From SHA-256
- Can't find two transactions with same hash

Commitment:

- Root commits to:
 - All transactions
 - Their content
 - Their order
- Can't change after PoW

Second preimage resistance:

- Can't find different tx with same proof path
- Would need to break hash function

Potential attacks and mitigations

Attack scenario:

- Malicious miner
- Includes invalid transaction
- Valid Merkle proof though!
- SPV client can't tell

Why it's hard:

- Full nodes reject invalid blocks
- Miner loses block reward
- Economic disincentive
- Need 51% to succeed

SPV best practices:

- Connect to multiple peers
- Wait for confirmations
- Check PoW chain is longest
- For large amounts: run full node

Security model

SPV trusts that majority hashpower is honest - same as full nodes!

Beyond Bitcoin: Merkle trees everywhere

Other blockchains:

- Ethereum: 3 Merkle trees per block
 1. Transactions
 2. Receipts
 3. State (accounts)
- Most cryptocurrencies use them

Beyond crypto:

- Git version control
- IPFS content addressing
- Certificate Transparency

We've seen them before!

- MLS (Secure Messaging)
- TreeKEM for group keys

Future variants:

- Verkle trees (smaller proofs)
- Sparse Merkle trees (state)
- Merkle-sum trees (amounts)

Universal primitive

Merkle trees: A fundamental tool for authenticated data structures

Example: Git is a blockchain!

Git commits are blocks:

- Each commit has:
 - Content (like transactions)
 - Timestamp
 - Hash of parent commit
 - Author signature
- Forms a chain back to initial commit
- Change any commit → all subsequent hashes change

Key differences:

- Git: Centralized (GitHub)
- Bitcoin: Decentralized (P2P)
- Git: No proof-of-work
- Bitcoin: Mining required
- Git: Trust repository owner
- Bitcoin: Trust no one

Same primitive

Both use Merkle trees and hash chains for tamper-evident history!

Section 2

Ethereum and Smart Contracts

Bitcoin's limitation: Just digital cash

Bitcoin can only:

- Send coins from A to B
- Simple spending conditions
- No loops or complex logic

Why?

- Intentionally limited scripting
- Security through simplicity
- One job: be money

What if we want more?

- Automated loans
- Decentralized exchanges
- Digital property
- Autonomous organizations

The vision

Programmable money with arbitrary logic!

Ethereum: Bitcoin + programming

What if the blockchain could run any program?

The idea (2013):

- Vitalik Buterin's insight
- Blockchain as "world computer"
- Store code, not just transactions
- Every node executes the code

Key innovation:

- Turing-complete language
- Persistent state storage
- Global consensus on computation
- Anyone can deploy code

Result

"Smart contracts" - code that runs trustlessly on the blockchain

From Bitcoin to Ethereum

Bitcoin:

- UTXO model (like cash)
- Limited scripting
- Tracks coin ownership
- Simple state transitions

Ethereum:

- Account model (like bank accounts)
- Full programming
- Tracks balances + code + data
- Complex state machine

Every transaction can trigger arbitrary code execution

Section 2

Ethereum and Smart Contracts

Subsection 2.1

Smart Contracts

What are smart contracts?

The vending machine:

- Insert \$1.50 → get soda
- Rules built into machine
- No human intermediary
- Can't cheat the machine

Smart contracts:

- Code on blockchain
- Rules in the code
- Executes automatically
- Can't be stopped or changed

Four key properties

Immutable · Deterministic · Autonomous · Transparent

Your first smart contract

```
pragma solidity ^0.8.0;

contract SimpleStorage {
    uint256 public number;

    function store(uint256 _num) public {
        number = _num;
    }

    function retrieve() public view returns (uint256) {
        return number;
    }
}
```

What this does:

- Stores one number
- Anyone can update it
- Anyone can read it
- Lives forever on blockchain

Language: Solidity

- Like JavaScript + types
- Compiles to bytecode
- Runs on EVM

How smart contracts work: The lifecycle

1. Deployment:

- Write code (Solidity)
- Compile to bytecode
- Send deployment transaction
- Contract gets address
- Now lives on blockchain!

2. Interaction:

- Send transaction to contract
- Call specific function
- All nodes execute code
- State changes recorded
- Everyone sees same result

Key insight

Once deployed, the code is **law**. No one can change it, not even the creator!

The gas mechanism: Paying for computation

Problem:

- What stops infinite loops?
- What prevents spam?
- How to price CPU time?

Solution: Gas

- Every operation costs gas
- Storage most expensive
- Simple math cheap
- User pays for execution

Gas prices:

- ADD: 3 gas
- MUL: 5 gas
- SSTORE: 20,000 gas

How it works:

- Set gas limit & price
- Fee = gas used $\times$ price
- Out of gas? Reverts!

Brilliance

Solves halting problem with economics!

How do you get gas? Understanding the gas market

You don't "buy" gas directly:

- Gas is just a unit of measurement
- Like "miles" or "kilowatts"
- You pay for gas in ETH

How it works:

- Have ETH in your account
- Send transaction
- Specify gas price you'll pay
- Miners/validators choose which txs to include

What sets gas prices?

- **Supply & demand!**
- Network congested? Prices up
- Quiet period? Prices down
- Users bid for block space

Auto-suggestion algorithm

Wallet automatically suggests gas price based on current market!

The gas auction: Priority fees

Post-EIP-1559 (2021):

- Base fee (burned)
 - Set by protocol
 - Adjusts with demand
- Priority fee (tip)
 - Goes to validator
 - You choose amount
 - Higher tip = faster inclusion

Fee calculation:

- Total fee = (Base fee + Priority fee) × Gas used
- Base fee: 30 gwei
- Your tip: 2 gwei
- Gas used: 21,000
- Cost: $(30 + 2) \times 21,000 = 672,000$ gwei = 0.000672 ETH

Market dynamics

High demand → Base fee increases

Want fast tx → Increase tip

A real example: Simple token

```
contract SimpleToken {
    mapping(address => uint256) public balances;

    constructor(uint256 initialSupply) {
        balances[msg.sender] = initialSupply;
    }

    function transfer(address to, uint256 amount) public returns (bool) {
        require(balances[msg.sender] ≥ amount, "Insufficient balance");
        balances[msg.sender] -= amount;
        balances[to] += amount;
        return true;
    }
}
```

Key concepts: msg.sender (who called this?), mapping (key-value storage), require (check conditions)

Decentralized Autonomous Organizations (DAOs)

What is a DAO?

- Organization governed by code
- Rules encoded in smart contracts
- No central authority
- Token holders vote on decisions
- Treasury controlled by members

How they work:

- Members hold governance tokens
- Proposals submitted on-chain
- Token-weighted voting
- Automatic execution if passed

Examples:

- MakerDAO: Manages DAI stablecoin
- Uniswap DAO: Protocol governance
- ConstitutionDAO: Attempted to buy US Constitution (\$47M raised!)

Use cases:

- Investment clubs
- Protocol governance
- Grant distribution
- Collective ownership

The DAO hack: When code goes wrong

June 2016:

- The DAO raises \$150M
- Largest crowdfund ever
- Decentralized VC fund
- All controlled by code

The attack:

- Reentrancy vulnerability
- Withdraw function flawed
- \$60M stolen in hours

What went wrong:

1. Call withdraw()
2. Contract sends ETH
3. Before updating balance...
4. Attacker calls withdraw again!
5. Repeat until drained

Lesson

Code is law... but bugs are expensive! Led to Ethereum hard fork.

The aftermath: Ethereum Classic

The controversial solution:

- Community debated: What to do?
- Funds locked, attacker waiting
- Hard fork proposed
- Rewrite history to undo hack
- Return stolen funds

The split (July 2016):

- Majority forked (Ethereum)
- Minority refused (Ethereum Classic)
- Same code, different chains
- Both still exist today

The philosophical divide:

- **Ethereum (ETH):**
 - “Code is not law if broken”
 - Pragmatic approach
 - Protect users
- **Ethereum Classic (ETC):**
 - “Code is law, no exceptions”
 - Immutability above all
 - No human intervention

The debate continues

Was the fork the right choice? Still controversial in crypto community!

Security matters: Common vulnerabilities

Top threats:

- **Reentrancy:** Like The DAO
- **Access control:** Who can call what?
- **Integer issues:** Over/underflow
- **Front-running:** Seeing txs before they execute

Best practices:

- Use OpenZeppelin libraries
- Check-Effects-Interactions pattern
- Comprehensive testing
- Professional audits
- Bug bounties

Remember

Can't patch code after deployment!

Real-world applications

DeFi (Decentralized Finance):

- Uniswap: Token swaps
- Aave: Lending/borrowing
- MakerDAO: Stablecoin

NFTs:

- Digital art ownership
- Gaming items
- Collectibles

Other uses:

- DAOs: Decentralized orgs
- ENS: Domain names
- Multi-sig wallets
- Prediction markets

The impact

\$50B+ locked in DeFi protocols!

NFTs: Non-Fungible Tokens

What are NFTs?

- Non-fungible = unique, not interchangeable
- Token standard (ERC-721)
- Proves ownership of digital asset
- Metadata points to content

How they work:

- Smart contract tracks ownership
- TokenID uniquely identifies each NFT
- Transfer ownership on-chain
- Anyone can verify authenticity

Use cases:

- Digital art (Beeple: \$69M sale)
- Profile pictures (Bored Apes)
- Gaming items
- Event tickets
- Domain names (ENS)

The controversy:

- Speculation bubble?
- Right-click save problem
- Environmental concerns (pre-Merge)
- What are you really buying?

Section 2

Ethereum and Smart Contracts

Subsection 2.2

Proof-of-Stake: The Merge

The energy problem

Ethereum's PoW:

- Like Bitcoin mining
- Massive energy use
- 78 TWh/year
- More than Austria!

The question:

- Can we secure network...
- Without burning energy?
- Still stay decentralized?

Proof-of-Stake idea:

- Replace miners with validators
- Stake ETH as collateral
- Get randomly selected
- Misbehave? Lose your stake!

Core insight

Economic security instead of energy security

The Merge: September 2022

One of the most ambitious upgrades in tech history

What happened:

- Swapped consensus engine
- While network was running
- \$200B+ at stake
- Zero downtime
- Like changing plane engines mid-flight!

Results:

- 99.95% energy reduction
- 900K+ validators
- \$40B+ staked
- 13-minute finality
- Network more secure than ever

From 13,000 households worth of energy to just 10

How PoS works

Become a validator:

- Stake 32 ETH ($\approx$ \$100K)
- Run validator software
- Stay online
- Act honestly

Rewards:

- $\approx$ 4-5% annual yield
- Block proposals
- Attestations

Penalties:

- Offline? Small penalties
- Double-sign? Slashed!
- Lose stake + ejected

Skin in the game

Can't fake having stake. Attack costs real money you'll lose!

What incentivizes honest behavior?

Carrots (rewards):

- Attestation rewards
 - Vote on correct chain head
 - ≈ 0.02 ETH per day
- Block proposal rewards
 - Randomly selected
 - Transaction fees + tips
 - Can be substantial
- Sync committee rewards
 - Help light clients
 - Additional yield

Sticks (penalties):

- Inactivity leaks
 - Offline? Lose rewards
 - Proportional to downtime
- Slashing
 - Double-sign? Lose 1+ ETH
 - Forced exit from validator set
 - Slashed funds burned
- Opportunity cost
 - Capital locked up
 - Acting honestly more profitable

Expected return: $\approx 4-5\%$ APR for honest validators

How is dishonesty detected?

The watchers:

- Every validator watches every other validator
- All attestations are public
- All blocks are public

Slashable offenses:

- **Double proposal:**
 - Propose 2 blocks at same height
 - Trying to fork the chain
- **Surround vote:**
 - Attest to conflicting checkpoints
 - Violates finality rules

Detection mechanism:

1. Validator misbehaves
2. Creates cryptographic evidence
 - Two signed messages
 - Both with validator's signature
 - Provably conflicting
3. Anyone can submit proof
4. Proof included in block
5. Slashing automatically executed

Critiques of Proof-of-Stake

Nothing at stake:

- In PoW, mining on multiple chains wastes energy
- In PoS, validating multiple chains is free
- Could validators hedge by validating all forks?
- Mitigation: Slashing for double-signing

Wealth concentration:

- Rich get richer (staking rewards)
- High barrier to entry (32 ETH)
- Centralization concern
- Counter: Staking pools, but adds trust

The debate

Is economic security as strong as energy security?

Section 2

Ethereum and Smart Contracts

Subsection 2.3

Scaling: Layer 2s

The blockchain trilemma

Pick any two:

Decentralization

Many validators

Security

Attack-resistant

Scalability

High throughput

The problem:

Ethereum: 15 TPS

Visa: 65,000 TPS

Gap: 4,000×

Why so slow?

Every node processes every transaction. Can't just use bigger servers!

Layer 2: The solution

Key insight:

- Not every node needs to execute every transaction
- Execute elsewhere
- Post results to Ethereum smart contract
- Prove correctness

L1 (Ethereum):

- Security layer
- Final settlement
- Data availability

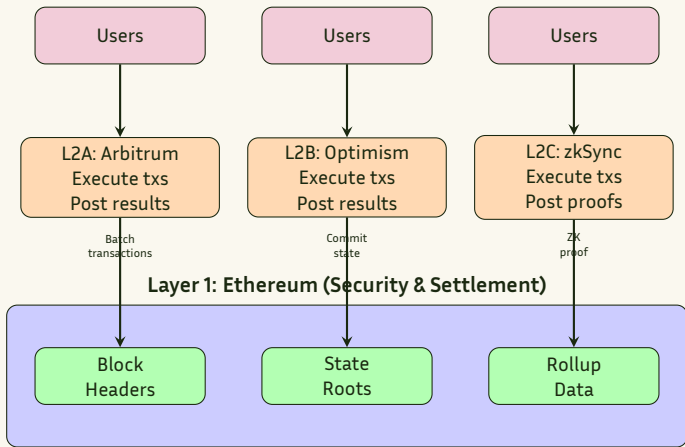
L2 (Rollups):

- Execute transactions
- Much cheaper
- Much faster
- Inherit L1 security

The result

100x cheaper fees, 100x higher throughput, same security!

Visualizing Layer 1 and Layer 2



Two types of rollups

Optimistic Rollups:

- Assume transactions valid
- 7-day challenge period
- Anyone can prove fraud
- If fraud found, rollback

Examples:

- Arbitrum (\$15B TVL)
- Optimism (\$7B TVL)
- Base (Coinbase's L2)

ZK Rollups:

- Cryptographic validity proofs
- Instant finality
- No waiting period
- Math proves correctness

Examples:

- zkSync
- StarkNet
- Polygon zkEVM

Both inherit Ethereum's security. Different trade-offs on speed vs complexity.

The L2 revolution

The numbers:

- L2s process 5-10× more than Ethereum
- Fees 10-100× cheaper
- 50+ L2s live or in development
- Transaction: \$0.10 vs \$10

Why this matters:

- Makes crypto actually usable
- Enables mass adoption
- Preserves decentralization
- No compromise on security

The future

Ethereum = settlement layer

L2s = execution layer

Users mostly interact with L2s

Challenges ahead

Current issues:

- Fragmentation (many L2s)
- Moving between L2s hard
- Centralized sequencers
- Complex for users

Being solved:

- Shared sequencers
- Fast bridges
- Better UX
- Decentralization roadmaps

The ecosystem is evolving fast. L2s are a path to scaling Ethereum while preserving what makes it special.



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.7: Cryptocurrency Cryptography

Nadim Kobeissi

<https://appliedcryptography.page>